# 19. Routing in React

## ▼ What is Routing in React JS?

- Routing means navigating from one page to another page.
- Routing in React means showing different pages (components) at different URLs without reloading the entire page.
- It enables Single Page Application (SPA) behavior.
- Basically, React focuses mainly on building the UI; routing is not built into the library, and it is provided by the **React Router** Library.

---

## ▼ Why Do We Need Routing in React?

- Without routing, you'd have to build separate HTML files for each page.

Imagine a normal website:

- `/` → Home Page
- `/about` → About Page
- `/contact` → Contact Page

Without routing:

- We **cannot navigate between these pages properly**
- We would have to write full-page logic manually
- User experience becomes slow

With routing:

- We can switch between pages **instantly**
- App behaves like a **real website**
- No full page reload
- Very fast performance

---

## ▼ Types of Routing in Web Applications

- There are two ways to add routing:

1. Client-side Routing (used in React)
2. Server-side Routing

## ▼ 1. Client-side Routing

- Client-side routing is one of the most important features to make use of while developing single-page web applications.
- Client-side routing means navigation happens inside the browser without requesting a new page from the server.
- Only the required component is rendered.
- URL changes, but page doesn't reload

---

## ▼ 2. Server-side Routing

- Server-side routing means every time the user changes the route, a request is sent to the server and a new HTML page is returned.
- Each route makes a **new request to the server**
- A **new HTML page is loaded every time**
- Browser **reloads on every navigation**
- Used in:
  - Traditional websites (HTML, PHP, JSP)
  - Backend frameworks (Node + Express)
  - SSR frameworks (Next.js, Nuxt.js)

---

# ▼ What is `React Router`?

- React Router is a popular JavaScript library for managing client-side routing in React applications.
- It allows us to navigate between different components (pages) without reloading the entire web page.
- It allows us to:
  - Define **routes**
  - Match **URLs**
  - Load the correct **components dynamically**

- It provides:
  1. `< BrowserRouter >`
  2. `< Routes >`
  3. `< Route >`
  4. `< Link >`
  5. `< NavLink >`
  6. Hooks like `useNavigate`, `useParams`, `useLocation`
- **Command to install the library:**

💡 npm install react-router-dom

- **React Router v6 is the most widely used stable version in production.**

---

# ▼ Ways to add Routing in React:

1. By Using `BrowserRouter` (Traditional / Simple Approach)
2. By Using `createBrowserRouter` (Advanced / Data APIs Approach)

---

# ▼ 1. By Using `BrowserRouter` (Traditional / Simple Approach)

- This is the **most popular and beginner-friendly method**.
- Used in **90% of React projects**.
- It utilizes the HTML5 history API to synchronize the UI with the URL, allowing navigation without full page reloads.
- **For Example →**

```
import React from "react";
import { BrowserRouter, Route, Routes } from "react-router-dom";
import Home from "./pages/Home";
import Login from "./pages/Login";
import Register from "./pages/Register";
```

```
import PageNotFound from "./pages/PageNotFound";

const App = () => {
  return (
    <>
      <BrowserRouter>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/login" element={<Login />} />
          <Route path="/register" element={<Register />} />
          <Route path="*" element={<PageNotFound />} />
        </Routes>
      </BrowserRouter>
    </>
  );
};

export default App;
```

- **Where**,
  1. `BrowserRouter`:
     a. It is like a **boss wrapper** for routing.
     b. `BrowserRouter` connects your React app with the browser URL.
  2. `Routes`:
     a. It is a **box that holds all your routes**.
     b. `Routes` is a container that keeps all route paths.
  3. `Route`:
     a. It connects a **URL path** with a **Component**.
     b. `Route` decides **which component should open on which URL**.
  4. `path`:
     a. It is the **URL name**.
  5. `element`:

a. `element` is the **component that will show on screen based on the path.**

6. `*`:

a. If the user types a **wrong URL.**

b. `*` means **any wrong or unknown URL**.

---

# ▼ 2. By Using `createBrowserRouter` (Advanced / Data APIs Approach)

- This is a **new modern routing system introduced in** `v6.4+`.
- Used for **large-scale & data-driven apps**.
- This is the recommended router for all React Router web projects.
- **For Example →**
- **Code: App.jsx**

```
import React from "react";
import { createBrowserRouter, RouterProvider } from "react-router-do
m";
import Home from "./pages/Home";
import Login from "./pages/Login";
import Register from "./pages/Register";
import Layout from "./pages/Layout";
import PageNotFound from "./pages/PageNotFound";

//creating the routes by using createBrowserRouter()
let router = createBrowserRouter([
  {
    path: "/",
    element: <Layout />,
    children: [
      {
        path: "/",
        element: <Home />,
      },
      {
```

```
      path: "/login",
      element: <Login />,
    },
    {
      path: "/register",
      element: <Register />,
    },
    {
      path: "*",
      element: <PageNotFound />,
    },
  ],
},
]);

const App = () => {
  return <RouterProvider router={router} />;
};

export default App;
```

- **Code: Navbar.jsx**

```
import React from "react";
import { Link } from "react-router-dom";

const Navbar = () => {
  return (
    <div className="font-sans p-4">
      <nav className="bg-gray-100 p-4 rounded-lg shadow-md mb-6">
        <ul className="flex gap-6 text-gray-800 font-medium">
          <li>
            <Link
              to="/"
              className="hover:text-blue-600 transition-colors duration-20
0"
            >
              Home
```

```
          </Link>
        </li>
        <li>
          <Link
            to="/login"
            className="hover:text-blue-600 transition-colors duration-20
0"
          >
            Login
          </Link>
        </li>
        <li>
          <Link
            to="/register"
            className="hover:text-blue-600 transition-colors duration-20
0"
          >
            Register
          </Link>
        </li>
      </ul>
    </nav>
  </div>
 );
};

export default Navbar;
```

- **Code: Layout.jsx**

```
import React from "react";
import Navbar from "./Navbar";
import { Outlet } from "react-router-dom";

const Layout = () => {
 return (
  <>
   <Navbar />
```

```
      <Outlet />
    </>
  );
};

export default Layout;
```

## ▼ 1. What is `createBrowserRouter()`?

- The `createBrowserRouter` is a **function** provided by the `react-router-dom` library, which is used to create a router object for React applications.
- It will accept an array of an object which consists of properties like path and element. (An Array of Route Objects)
- **Syntax:** `createBrowserRouter([{}])`

---

## ▼ 2. What is `<RouterProvider/>`?

- `RouterProvider` is the **connector** that connects your React app with the router.
- **Syntax:**

```
import { RouterProvider } from "react-router-dom";

<RouterProvider router={router} />
```

---

## ▼ 3. What is `router` in RouterProvider?

- `router` is a **prop** or **attribute** for the RouterProvider component, which will accept a routes object.

---

## ▼ 4. What are `children` in createBrowserRouter?

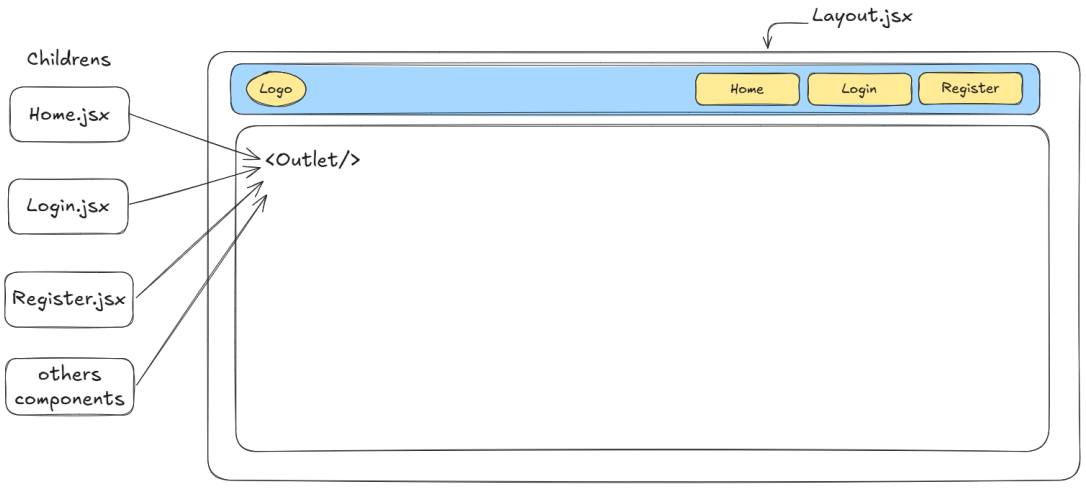
- `children` is an array of **nested routes** under a parent route.
- `children` are the **sub-pages inside a main page**.
- And these children are also accepted as an array of object.
- These child components are rendered inside the parent route's layout using `<Outlet/>`.

- **For Example →**

```
const router = createBrowserRouter([
  {
    path: "/dashboard",
    element: <Dashboard />,
    children: [
      { path: "profile", element: <Profile /> },
      { path: "settings", element: <Settings /> }
    ]
  }
]);
```

## ▼ 5. What is `<Outlet/>` Component?

- The `<Outlet/>` component is a placeholder within a parent route's component that tells React Router where to render the child routes.
- `<Outlet/>` is the **empty space where child pages will appear**.




# ▼ What is `Link` in React Router?

- `Link` is used to go from one page to another in a React app without reloading the page.
- Internally, it will be converted into the `<a>` tag of HTML.
- **Syntax**:

```
import { Link } from "react-router-dom";

<Link to="/login">Login</Link>
```

---

# ▼ What is `NavLink` in React Router?

- `NavLink` is a special type of `Link` that tells which page is currently active.
- It's similar to the `< Link>` component but provides additional functionality for indicating the ***active link***.
- **Syntax**:

```
import { NavLink } from "react-router-dom";

<NavLink to="/">Home</NavLink>
<NavLink to="/about">About</NavLink>
```

## ▼ Props

- The `<NavLink>` components come with props (properties), these props can be used to style the `<NavLink>` component which determines how NavLink is rendered on the page.
- **These includes:**
  - to
  - active (can be specified but exist by default)
  - style
  - className
  - children
  - **end**
- **How to see the props:**
- **Code:**

```
<NavLink
    to="/"
```

```
    className={(props) => {
        console.log("props:", props);
    }}>
      Home
</NavLink>
```

- **Output:**

```
props:  ▾ {isActive: true, isPending: false, isTransitioning: false} i
            isActive: true
            isPending: false
            isTransitioning: false
          ▸ [[Prototype]]: Object
```

- `isActive`:
  - `isActive: true` ⇒ means **you are currently on that link's route/page**.
  - `isActive: false` ⇒ means **you are not on that link's page**.
- `isPending`:
  - `isPending: true` ⇒ means **you clicked a link and the next page is loading** (React Router is preparing the route, maybe fetching data).
  - `isPending: false` ⇒ means **no new route is being loaded at the moment**.
- `isTransitioning`:
  - `isTransitioning: true` ⇒ means **the route change has started and is actively in progress** (like animation, data fetching, or page update).
  - `isTransitioning: false` ⇒ means **no route transition is happening**.

---

## ▼ How to provide the `isActive` class

- **Syntax: Destructor the `isActive` class inside the callback function inside the className.**

```
<NavLink to="/" className={({isActive}) => isActive ? "style-1" : "style-2"}>Home</NavLink>
```

---

## ▼ What is the `end` in <NavLink>?

- The `end` prop is used to indicate the route where the ***active*** or ***pending*** state should stop.
- It prevents the children from inheriting the active state from the parent `<NavLink>`.
- The `end` prop in `< NavLink >` is used to **prevent partial matching** of routes.

---

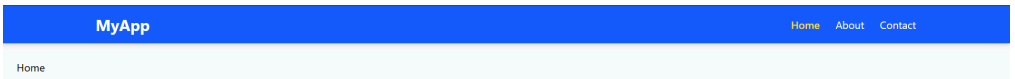
## ▼ Example by using `<NavLink>`

- **Code: Navbar.jsx**

```
import React from "react";
import { NavLink } from "react-router-dom";

const Navbar = () => {
  return (
    <nav className="bg-blue-600 shadow-md">
      <div className="max-w-7xl mx-auto px-4 py-3 flex justify-betw
een items-center">
        <div className="text-white text-2xl font-bold">MyApp</div>
        <ul className="flex space-x-6">
          <li>
            <NavLink
              to="/"
              className={({ isActive }) =>
                isActive
                  ? "text-yellow-300 font-semibold"
                  : "text-white hover:text-yellow-300 transition duration-20
0"
              }
            >
              Home
            </NavLink>
          </li>
          <li>
            <NavLink
```

```
          to="/about"
          className={({ isActive }) =>
            isActive
              ? "text-yellow-300 font-semibold"
              : "text-white hover:text-yellow-300 transition duration-20
0"
          }
        >
          About
        </NavLink>
      </li>
      <li>
        <NavLink
          to="/contact"
          className={({ isActive }) =>
            isActive
              ? "text-yellow-300 font-semibold"
              : "text-white hover:text-yellow-300 transition duration-20
0"
          }
        >
          Contact
        </NavLink>
      </li>
    </ul>
  </div>
</nav>
);
};

export default Navbar;
```

- **Output: When Home is active**




- **Output: When Contact is active**

# ▼ Nested Routing (Layout System)

- Nested Routing means routing inside another route.
- **For Example →**

```
/dashboard
/dashboard/profile
/dashboard/settings
```

- **Example →**

```
const router = createBrowserRouter([
  {
    path: "/dashboard",
    element: <Dashboard />,
    children: [
      { path: "profile", element: <Profile /> },
      { path: "settings", element: <Settings /> }
    ]
  }
]);
```

-

# ▼ Dynamic Routing

- Dynamic routing means the URL changes dynamically according to the id or name.
- **For Example**: Github

```
/product/5
/user/101
```

- Here `5` and `101` are **dynamic values**.

- **For Example →**

```
path: "/product/:id",
element: <Product/>
```

## ▼ `useParams()` Hook

- The **useParams()** hook is used to read the dynamic value from the URL.
- This hook returns an **object** containing the parameters as key-value pairs.
- Syntax:

```
const params = useParams();
```

- **For Example →**

```
import { useParams } from "react-router-dom";

function Product() {
  const { id } = useParams();

  return <h1>Product ID: { id }</h1>;
}
```

## ▼ `useNavigate()` Hook

- The `useNavigate` hook is a new addition to React Router 6.
- The `useNavigate` is used to change pages using JavaScript instead of clicking a link.
- It is used **only in functional components.**
- It helps in **programmatic navigation** (navigation using code)
- Once you have React Router 6 installed, you can import the `useNavigate` hook from the `react-router-dom` package:

💡 import { useNavigate } from 'react-router-dom';

- **Syntax:**

```
navigate(path, {
  replace: false,     // optional, default is false
  state: { key: "value" } // optional, sends state to target route
});
```

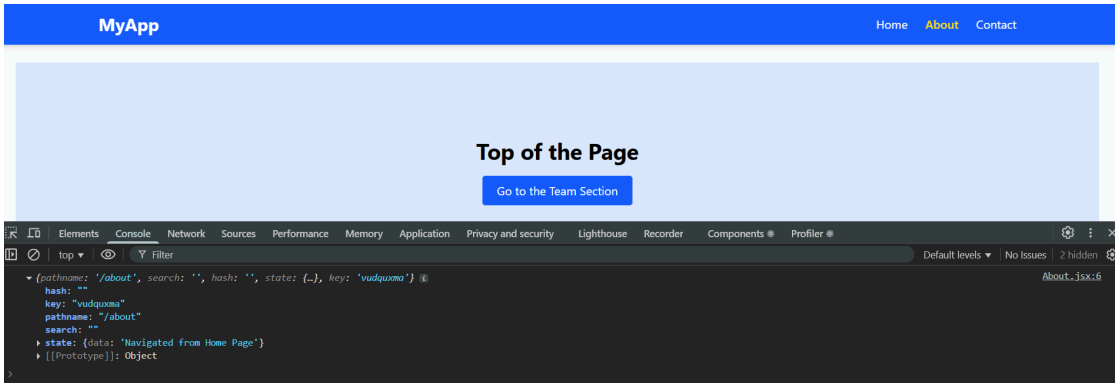
## ▼ How to use `useNavigate` in your React application

- **Code: Home.jsx**

```
import React from "react";
import { useNavigate } from "react-router-dom";

const Home = () => {
  let navigate = useNavigate();

  let handleRedirect = () => {
    // navigate("/about");
    navigate("/about", {
      state: { data: "Navigated from Home Page" },
      replace: true,
        });
  };
  return (
    <div>
      <h1 className="p-2">Home</h1>
      <button
        onClick={handleRedirect}
        className="px-6 py-2 bg-blue-600 rounded-md text-white"
      >
        Go to about
      </button>
    </div>
  );
```

```
};

export default Home;
```

- **You can access that state coming from the `Home.jsx` by using the `useLocation` Hook.**
- **Output:**




## ▼ `useLocation()` Hook

- The `useLocation` hook in React Router is a tool that provides access to the current URL's location object.
- The `useLocation` hook in React Router is used to return the current location of a React component.
- The `useLocation` returns the current location as an object and comes with props such as
  - pathname
  - state
  - search
  - key
  - hash
- **For Example →**

```
let location = useLocation();
```

```
console.log(location); //returns the location object
```

- **Output:**

```
▾ {pathname: '/about', search: '', hash: '', state: null, key: 'pueznp1k'} ℹ
    hash: ""
    key: "pueznp1k"
    pathname: "/about"
    search: ""
    state: null
  ▸ [[Prototype]]: Object
```

- ***Important**: The* `useLocation()` *hook must be used inside a component that is a child of a Router component.*
- *If you try to use* `useLocation()` *outside that context, you will get this error:*

```
Uncaught Error: useLocation() may be used only in the context of a <Ro
uter>
```

---

## ▼ pathname

- The `pathname` property (prop) from the `location` object is used to return the path name of a given component. The path name is the name that comes after the port number.

```
                    //pathname
                        👇
http://localhost:5173/products
                    👆
            //port number
```

- **For Example →**
- **Code: About.jsx**

```
import React from "react";
import { useLocation } from "react-router-dom";

const About = () => {
  let location = useLocation();
```

```
  console.log(location);
  //  or let {pathname} = useLocation();
  let pathname = location.pathname;
  console.log(pathname);
  return <div>About</div>;
};

export default About;
```

- **Output:**

```
▸ {pathname: '/about', search: '', hash: '', state: null, key: 'pueznp1k'}
/about
```

## ▼ state

- The `state` property (prop) from the `location` object is used to set data on parent components that can be accessed by the children components.
- **For Example →**
- **Code: Inside Navbar.jsx**

```
<Link
    to="/about"
    state={"Message coming from the navbar through state"}
    className="text-white hover:text-yellow-300 transition duration
-200">
    About
</Link>
```

- **Code: About.jsx**

```
import React from "react";
import { useLocation } from "react-router-dom";

const About = () => {
  let location = useLocation();
  console.log(location);
```

```
  let state = location.state;
  console.log("state:", state);
  return <div>About</div>;
};

export default About;
```

- **Output:**

```
About.jsx:6
▸ {pathname: '/about', search: '', hash: '', state: 'Message coming from the navbar through state', key: '4u3qh8vy'}
state: Message coming from the navbar through state        About.jsx:8
```